

Sentiment-Fake-News-Analysis

Supervised Learning Course Project

Project Report

Team Information

Name	ID
Mohamed Beshr Al Sofi	20230717
Essam Mamdouh Saad	20230626
Eman Emad Farghaly	20230619
Malak Mohamed Saad	20230417

A complete supervised learning pipeline for Twitter sentiment analysis, comparing classical TF-IDF-based models with deep convolutional, gated recurrent, and vanilla recurrent neural networks.

Phase 1 — Data Processing & Classical ML

1. Introduction

This phase focuses on building a strong baseline for sentiment analysis using classical Natural Language Processing (NLP) techniques. The main objectives are:

- Cleaning and preprocessing raw text data
- Performing exploratory data analysis (EDA)
- Converting text into numerical features using TF-IDF
- Training classical machine learning models
- Evaluating and comparing model performance

2. Dataset Description

We used the **Sentiment140** dataset, which contains 1.6 million tweets labelled automatically based on emoticons.

Column	Description
target	Sentiment label (0 = Negative, 4 = Positive)
id	Tweet ID
date	Date of tweet
flag	Query
user	Username
text	Tweet content

Labels were remapped: 0 -> Negative (0) and 4 -> Positive (1). The resulting dataset is approximately balanced (~50% per class).

3. Data Cleaning & Preprocessing

The dataset required extensive preprocessing to prepare it for modelling.

Steps Applied

- Lowercasing text
- Removing URLs, mentions (@user), hashtags (#topic), and HTML entities
- Removing non-alphabetic characters
- Tokenization
- Stopword removal (NLTK English stopwords)
- Stemming with the Porter Stemmer

Additional Features Created

- Tweet length (characters)
- Word count
- Clean word count (post-preprocessing)

Result: ~18K duplicate tweets removed; the cleaned dataset is ready for feature extraction.

4. Exploratory Data Analysis (EDA)

- **Sentiment distribution:** balanced ~50% Negative / 50% Positive
- **Tweet length:** average ~74 characters; most tweets are short (typical for Twitter)
- **Top words (Negative):** miss, want, feel
- **Top words (Positive):** love, good, thank
- **Time analysis:** tweets distributed across all hours; no strong correlation with sentiment
- **Correlation:** weak correlation between numeric features and sentiment

5. Feature Engineering — TF-IDF

Text was converted into numerical format using a **TF-IDF vectorizer** with `ngram_range = (1, 2)`, producing a sparse high-dimensional matrix suitable for classical ML models.

6. Model Training

6.1 K-Nearest Neighbours (KNN)

Challenge: KNN is computationally expensive on large datasets. **Solution:** TruncatedSVD (LSA) reduced features to 100 components. **Result:** Accuracy ~0.65.

6.2 Multinomial Naive Bayes

Very efficient for text data; works well with TF-IDF. **Result:** Accuracy ~0.75 — the best Phase 1 classifier; handles high-dimensional sparse data well.

6.3 Random Forest

Applied on SVD-reduced features. **Result:** Accuracy ~0.70. More robust than KNN; slower training; did not outperform Naive Bayes.

6.4 XGBoost

Gradient boosting on SVD features. **Result:** Accuracy ~0.70. Strong model but similar performance to Random Forest; sensitive to tuning.

7. Hyperparameter Tuning

RandomizedSearchCV was used for speed. Tuning did not significantly improve performance — default parameters were already near-optimal.

8. Model Comparison

Model	Accuracy	F1 Score
KNN	~0.6511	~0.6532
Naive Bayes	~0.75	~0.75

Model	Accuracy	F1 Score
Random Forest	~0.70	~0.70
XGBoost	~0.70	~0.70

9. Key Insights & Challenges

- Naive Bayes is the best classical baseline for TF-IDF features.
- KNN is inefficient and does not scale to 1.6M tweets without aggressive dimensionality reduction.
- Ensemble models (Random Forest, XGBoost) are powerful but slower and tuning-sensitive.
- Dimensionality reduction (TruncatedSVD) is essential for scalability.
- Hyperparameter tuning offers only marginal gains over sensible defaults.

Challenges Faced

- Large dataset size (1.6M tweets) caused high memory usage during SVD
- Long training time for ensemble models
- Managing computation efficiently within Kaggle's resource limits

10. Conclusion — Phase 1

We successfully built a complete classical NLP pipeline: cleaned and processed large-scale text data, built multiple ML models, and compared them rigorously. **Naive Bayes** achieved the best Phase 1 results, providing a strong baseline for the deep-learning phases that follow.

Phase 2 — Deep Learning (CNN + Autoencoder)

1. Introduction

This phase builds deep learning models for sentiment analysis to improve upon the classical ML baselines from Phase 1. Objectives:

- Convert text to integer sequences (tokenization & padding)
- Learn word embeddings (random initialization & pre-trained GloVe)
- Train Convolutional Neural Network (CNN) models
- Explore an Autoencoder for unsupervised feature learning
- Compare optimizers, dropout rates, and embedding strategies
- Evaluate Phase 2 models against Phase 1 baselines

2. Tokenization & Padding

Cleaned text was converted into integer sequences using a custom word-to-index vocabulary built from the training corpus:

Step	Description
1	Build a word frequency dictionary from all cleaned tweets
2	Create vocabulary mapping (word -> integer index)
3	Convert each tweet into a sequence of integer indices
4	Post-pad / truncate to a fixed SEQ_MAX_LEN

3. Word Embeddings

Strategy	Description
Random Initialization	Embedding weights learned from scratch during training
GloVe Frozen	Pre-trained 200d GloVe vectors loaded; <code>trainable = False</code>
GloVe Trainable	Pre-trained 200d GloVe vectors loaded; fine-tuned during training

- Pre-trained GloVe vectors provide semantic word representations.
- Embeddings can be frozen (fixed) or trainable (fine-tuned).
- Trainable embeddings let the model adapt vectors to the sentiment task.

4. CNN Architecture

Layer	Description
Embedding	Converts integer sequences to dense vectors

Layer	Description
Conv1D	Extracts local n-gram features from the text
GlobalMaxPooling1D	Reduces each feature map to its maximum value
Dense (hidden)	Fully connected layer for classification
Dropout	Regularization to prevent overfitting
Dense (output)	Sigmoid activation for binary classification

Training: Binary Crossentropy loss; Adam optimizer (baseline) with RMSprop and SGD+Momentum variants; up to 10 epochs with EarlyStopping.

5. Experiments & Hyperparameter Tuning

5.1 Dropout Comparison

Variant	Dropout	Accuracy	F1
CNN GloVe frozen, Adam, drop=0.5	0.5	0.7462	0.7435
CNN GloVe frozen, Adam, drop=0.3	0.3	0.7482	0.7523

Insight: Reducing dropout from 0.5 to 0.3 slightly improved performance, suggesting the baseline model was over-regularized.

5.2 Embedding Strategy Comparison

Variant	Accuracy	F1
GloVe Frozen	0.7462	0.7435
GloVe Trainable	0.7807	0.7816
No Pre-trained Embeddings (Random)	0.7806	0.7808

Insight: Trainable GloVe and random initialization perform similarly, both significantly outperforming frozen GloVe. Allowing the embedding layer to fine-tune is critical.

5.3 Optimizer Comparison

Architecture fixed: CNN with frozen GloVe, dropout = 0.5.

Optimizer	LR	Accuracy	F1
Adam	1e-3	0.7462	0.7435
RMSprop	1e-3	0.7492	0.7519
SGD + Momentum	1e-2, mom=0.9	0.7477	0.7536

Insight: All three optimizers performed comparably. SGD+Momentum achieved the highest F1 with frozen embeddings, but the differences are marginal.

5.4 Autoencoder

An autoencoder was explored for unsupervised feature learning: it encodes tweets into a compressed latent representation and decodes back to the original. The encoder portion can be used as a feature extractor.

Insight: Provided an alternative perspective on representation learning, but supervised CNN models clearly outperformed the autoencoder-based pipeline.

6. Model Evaluation & Phase 2 Comparison

CNN training behaviour: early stopping triggered around epoch 8 (best validation loss at epoch 5); training curves converge within the first few epochs and validation metrics plateau quickly.

Model	Accuracy	F1
CNN GloVe trainable, Adam, drop=0.5	0.7816	0.7825
CNN No Pretrained Emb (random init)	0.7789	0.7820
CNN GloVe frozen, RMSprop	0.7492	0.7547
CNN GloVe frozen, Adam, drop=0.3	0.7482	0.7538
CNN GloVe frozen, SGD + Momentum	0.7462	0.7512
CNN GloVe frozen, Adam, drop=0.5	0.7462	0.7435

7. Phase 1 vs Phase 2 — Cross-Phase Comparison

Criterion	Phase 1 - Classical ML	Phase 2 - Deep Learning
Best Accuracy	~0.6511	~0.7816
Best F1	~0.6532	~0.7825
Training Time	Minutes (CPU)	Minutes-Hours (GPU)
Inference Speed	Very fast	Fast (batched)
Interpretability	High (feature weights)	Low (black-box)
Requires GPU	No	Recommended
Handles OOV Words	No (fixed vocab)	Yes (embeddings)
Scales to More Data	Limited (memory)	Excellent

8. Key Insights — Phase 2

- Deep learning (CNN) models significantly outperform classical ML baselines.
- **Trainable embeddings are crucial** — frozen GloVe underperforms trainable variants by ~3pt.
- Random initialization performs on par with pre-trained GloVe when fine-tuning is allowed.
- Optimizer choice has minimal impact when embeddings are frozen.
- Early stopping is effective at preventing overfitting in CNN training.
- Deep learning scales better with data but requires more compute resources.

9. Challenges & Conclusion — Phase 2

- High computational cost of training CNN models on 1.6M tweets
- GPU memory management during embedding and training

- Balancing model complexity with available compute
- Determining optimal sequence length for padding
- Managing multiple experiment variants and tracking results

Conclusion: CNN with trainable GloVe embeddings achieved the best Phase 2 results (Accuracy = **0.7816**, F1 = **0.7825**), a ~20% improvement over the best Phase 1 model.

Phase 3 — Recurrent Models (GRU, LSTM, Vanilla RNN)

1. Motivation

CNNs detect local n-gram patterns through convolution but do not model how words flow over time. Recurrent networks process the tweet token-by-token while maintaining a hidden state, which makes them a natural fit for sentiment tasks where word order and negation cues matter. We compare three recurrent cell types:

- **GRU** — gated cell with update / reset gates; mitigates the vanishing-gradient problem of plain RNNs while being lighter than LSTM.
- **LSTM** — the original gated cell with input, forget, and output gates; explicitly maintains a long-term cell state alongside the hidden state.
- **SimpleRNN** — classic Elman-style update with a single weight matrix and tanh nonlinearity; the textbook baseline that gated cells improve on.

2. GRU Model

2.1 Architecture

Layer	Configuration
Embedding	vocab_size x 200, GloVe weights, trainable = False
GRU	units = 128, dropout = 0.2, recurrent_dropout = 0.2
Dropout	rate = 0.5
Dense	64 units, ReLU
Dense (output)	1 unit, sigmoid

2.2 Training Configuration

Setting	Value
Loss	Binary Crossentropy
Optimizer	Adam (learning rate = 1e-3)
Epochs	10 (EarlyStopping, patience = 3, restore_best_weights = True)
Batch size	256
Split	60% train / 20% val / 20% test (lowercase X_tr/X_val/X_te)
Time / epoch	~220 s (GPU)

Training: val_loss fell from **0.5086** at epoch 1 to **0.4864** at epoch 8; EarlyStopping restored the best weights.

2.3 Test-Set Results

Metric	Negative	Positive	Overall
Precision	0.77	0.75	0.7529
Recall	0.74	0.78	0.7804
F1	0.76	0.77	0.7664
Support	157,604	158,690	316,294
Accuracy			0.7613

Insight: The GRU outperforms every frozen-GloVe CNN variant on F1. Its slight bias toward the Positive class (recall 0.78 vs 0.74) is consistent with the data distribution.

3. LSTM Model

3.1 Architecture

Layer	Configuration
Embedding	vocab_size x 200, GloVe weights, trainable = True
LSTM	units = 64, dropout = 0.2, return_sequences = False
Dropout	rate = 0.4
Dense	64 units, ReLU
Dropout	rate = 0.3
Dense (output)	1 unit, sigmoid

Key difference vs the GRU: the LSTM uses **trainable** embeddings (the embedding layer fine-tunes during training) and a smaller recurrent state (64 vs 128).

3.2 Training Configuration

Setting	Value
Loss	Binary Crossentropy
Optimizer	Adam (learning rate = 1e-3, default)
Callbacks	EarlyStopping(patience = 3) + ReduceLROnPlateau(factor = 0.5, patience = 1)
Epochs	10 (stopped after 5 by EarlyStopping)
Batch size	32
Split	60% train / 20% val / 20% test (uppercase X_train/X_val/X_test)
Time / epoch	~250 s (GPU)

3.3 Training Trajectory

Epoch	Train Acc	Val Acc	Train Loss	Val Loss	LR
1	0.7520	0.7811	0.5060	0.4586	1e-3
2	0.7875	0.7836	0.4530	0.4548	1e-3
3	0.7976	0.7841	0.4357	0.4567	1e-3
4	0.8087	0.7834	0.4167	0.4592	5e-4
5	0.8165	0.7837	0.4016	0.4646	2.5e-4

Best **val_loss** achieved at epoch 2 (0.4548). Training accuracy keeps climbing (0.7520 -> 0.8165) while validation accuracy plateaus around 0.78 — classic overfitting onset, correctly caught by EarlyStopping.

3.4 Test-Set Results

Metric	Negative	Positive	Overall
Precision	0.77	0.79	0.78 (macro)

Metric	Negative	Positive	Overall
Recall	0.80	0.77	0.78 (macro)
F1	0.79	0.78	0.7793
Support	157,604	158,690	316,294
Accuracy			0.7824

Confusion matrix: True Negatives = 125,945; False Positives = 31,659; False Negatives = 37,178; True Positives = 121,512. The model is slightly biased toward predicting Negative (FN > FP).

Insight: The LSTM matches the best CNN (trainable GloVe) on accuracy (0.7824 vs 0.7816) and is just behind on F1 (0.7793 vs 0.7825). With trainable embeddings, gated recurrence is competitive with the best convolutional model.

4. Vanilla RNN (SimpleRNN)

4.1 Architecture

Layer	Configuration
Embedding	vocab_size x 200, GloVe weights, trainable = False
SimpleRNN	units = 128, dropout = 0.2, recurrent_dropout = 0.2
Dropout	rate = 0.5
Dense	64 units, ReLU
Dense (output)	1 unit, sigmoid

Same depth and hyperparameters as the GRU; only the recurrent cell type changes. This isolates the effect of gating mechanisms.

4.2 Test-Set Results

Metric	Negative	Positive	Overall
Precision	0.00	0.50	0.25 (macro)
Recall	0.00	1.00	0.50 (macro)
F1	0.00	0.67	0.6682
Support	157,604	158,690	316,294
Accuracy			0.5017

Failure mode: Training loss stagnated near $\ln(2) \approx 0.6931$ (val_loss = 0.6932 at epoch 1, unchanged at the start of epoch 2). The model collapsed to predicting the majority class (Positive) for every tweet, hence accuracy $\approx 50\%$ and recall = 1.00 / 0.00.

Why this happens: SimpleRNN suffers from **vanishing gradients** on sequences of SEQ_MAX_LEN tokens. Combined with frozen embeddings that the RNN cannot fine-tune, the gradient signal is too weak to escape the trivial constant-prediction solution, and EarlyStopping triggers before any real learning happens.

4.3 Mitigations (Future Work)

- Allow the embedding layer to fine-tune (`trainable = True`) so the RNN gets a learnable signal at the input.
- Apply gradient clipping (`clipnorm = 1.0`) and a lower learning rate ($1e-4$).
- Reduce sequence length or use a smaller hidden state (e.g. 64 units).
- In practice, prefer GRU / LSTM — the failure observed here is exactly the historical motivation for gated recurrent units.

5. Recurrent Models — Side-by-Side

Model	Embedding	Hidden	Accuracy	F1
GRU (Adam)	GloVe frozen	128	0.7613	0.7664
LSTM (Adam, RLR)	GloVe trainable	64	0.7824	0.7793
SimpleRNN (Adam)	GloVe frozen	128	0.5017	0.6682

Insight: Direct apples-to-apples comparison is partial: GRU and SimpleRNN share the (lowercase) X_tr / X_te split, while the LSTM was run on a separately-seeded (uppercase) X_train / X_test split. Both splits use random_state = 42 on the same underlying matrix, so the metrics are closely comparable but not bit-exact.

6. Updated Global Leaderboard (by F1)

Rank	Model	Phase	Accuracy	F1
1	CNN GloVe trainable, Adam, drop=0.5	2	0.7816	0.7825
2	CNN No Pretrained Emb (random init)	2	0.7789	0.7820
3	LSTM GloVe trainable, Adam + RLR	3	0.7824	0.7793
4	GRU (GloVe frozen, Adam)	3	0.7613	0.7664
5	CNN GloVe frozen, RMSprop	2	0.7482	0.7547
6	CNN GloVe frozen, Adam, drop=0.3	2	0.7482	0.7538
7	CNN GloVe frozen, SGD + Momentum	2	0.7462	0.7512
8	CNN GloVe frozen, Adam, drop=0.5	2	0.7475	0.7487
9	SimpleRNN (GloVe frozen, Adam)	3	0.5017	0.6682
10	Naive Bayes (TF-IDF)	1	~0.75	~0.75
11	Random Forest	1	~0.70	~0.70
12	XGBoost	1	~0.70	~0.70
13	KNN	1	0.6511	0.6532

7. Key Insights

- **Gating matters.** Replacing the recurrent cell from SimpleRNN to GRU (with everything else fixed) lifts accuracy from **0.5017** to **0.7613** — a +26pt jump.
- **Trainable embeddings + gated recurrence is the recipe.** The LSTM with trainable GloVe reaches 0.7824 accuracy, within 0.1pt of the best CNN.
- **Recurrent > convolutional under frozen embeddings.** The GRU beats every frozen-GloVe CNN on F1; sequential context pays off when the embedding layer can't adapt.
- **Compute trade-off.** LSTM (~250s/epoch, batch 32) and GRU (~220s/epoch, batch 256) are both expensive; SimpleRNN is faster (~36s/epoch) but does not converge here.

- **Practical recommendation:** use GRU or LSTM as the default recurrent baseline; reserve SimpleRNN only for pedagogical demonstrations of the vanishing-gradient problem.

8. Challenges & Final Conclusion

- Long per-epoch training time on GRU / LSTM (~3.7-4.2 minutes / epoch × 5-10 epochs on 1.6M tweets).
- Diagnosing the SimpleRNN collapse required inspecting the classification report — epoch-1 loss alone looked indistinguishable from a working model.
- Maintaining apples-to-apples comparability across CNN / GRU / LSTM / RNN required tracking which sections share which train/val/test split.

Final conclusion: Across three phases, the project covers the full arc from classical TF-IDF baselines (best F1 ~0.75) to deep convolutional and recurrent neural networks (best F1 0.7825). The two top models — CNN with trainable GloVe (F1 = **0.7825**) and LSTM with trainable GloVe (Acc = **0.7824**) — converge on the same lesson: **letting the embedding layer fine-tune** is the single most impactful architectural choice on Sentiment140.